

Italian Restaurant Sales Analytics

01 - Sales Data Cleaning and Preparation

Objective

This notebook cleans and validates the **Sales** worksheet from the Italian Restaurant Sales Analytics dataset. It verifies data quality, checks for missing values and duplicates, validates data types, and exports a cleaned dataset for further analysis in Python, SQL, and Power BI.

Step 1 - Import Libraries

The first step is to import the Python libraries required for data manipulation and analysis.

```
In [6]: # Import pandas for data manipulation and analysis
import pandas as pd
```

Step 2 - Load the Excel Workbook

The next step is to define the location of the Excel file and load the workbook into Python.

```
In [7]: # Define the relative path to the Excel workbook
file_path = "../data/raw/restaurant-sales-analytics-data.xlsx"
```

Step 3 - Read the Sales Worksheet

Load the Sales worksheet into a pandas DataFrame.

Installing Required Dependencies (If Needed)

This project reads Excel files using the `openpyxl` engine. If this library is not installed in your Python environment, run the following cell once before continuing.

Note: If `openpyxl` is already installed, you can skip this step.

```
In [8]: %pip install openpyxl
```

```
Requirement already satisfied: openpyxl in c:\users\guido acosta\appdata\local\progr
ams\python\python313\lib\site-packages (3.1.5)
Requirement already satisfied: et-xmlfile in c:\users\guido acosta\appdata\local\pro
grams\python\python313\lib\site-packages (from openpyxl) (2.0.0)
Note: you may need to restart the kernel to use updated packages.
```

```
[notice] A new release of pip is available: 25.3 -> 26.2.1
[notice] To update, run: python.exe -m pip install --upgrade pip
```

```
In [9]: # Load the Sales worksheet into a pandas DataFrame
sales = pd.read_excel(file_path, sheet_name="Sales")
```

Step 4 - Preview the Dataset

Display the first five rows of the dataset.

```
In [10]: # Display the first five rows
sales.head()
```

```
Out[10]:
```

	Date	Category	Sales
0	2025-03-01	Food	74471.064
1	2025-03-01	Beverages	12902.220
2	2025-03-01	Events	1957.200
3	2025-03-01	Liquor	14545.870
4	2025-04-01	Food	61905.340

Step 5 - Inspect the Dataset Structure

Display the structure of the dataset, including data types, number of non-null values, and memory usage.

```
In [11]: # Display dataset information
sales.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 52 entries, 0 to 51
Data columns (total 3 columns):
#   Column      Non-Null Count  Dtype
---  -
0   Date        52 non-null    datetime64[ns]
1   Category    52 non-null    object
2   Sales       52 non-null    float64
dtypes: datetime64[ns](1), float64(1), object(1)
memory usage: 1.3+ KB
```

Step 6 - Check Dataset Dimensions

Display the number of rows and columns in the dataset.

```
In [12]: # Display dataset dimensions
sales.shape
```

Out[12]: (52, 3)

Step 7 - Check Missing Values

Identify whether the dataset contains any missing values in each column.

```
In [13]: # Check for missing values
sales.isnull().sum()
```

```
Out[13]: Date          0
         Category      0
         Sales         0
         dtype: int64
```

Step 8 - Check for Duplicate Rows

Identify whether the dataset contains duplicate records that may affect the analysis.

```
In [14]: # Check for duplicate rows
sales.duplicated().sum()
```

```
Out[14]: np.int64(0)
```

Step 9 - Display Column Names

Display all column names in the dataset.

```
In [15]: # Display column names
sales.columns
```

```
Out[15]: Index(['Date', 'Category', 'Sales'], dtype='object')
```

Step 10 - Display Summary Statistics

Display descriptive statistics for the numerical columns to better understand the distribution of the data.

```
In [16]: # Display summary statistics
sales.describe()
```

Out[16]:

	Date	Sales
count	52	52.000000
mean	2025-08-31 11:04:36.923076864	25861.678538
min	2025-03-01 00:00:00	1343.600000
25%	2025-06-01 00:00:00	10126.677500
50%	2025-09-01 00:00:00	14348.690000
75%	2025-12-01 00:00:00	31729.735000
max	2026-03-01 00:00:00	86570.790000
std	NaN	28197.244924

Step 11 - Check Unique Categories

Display the unique values in the Category column to verify that all expected categories are present.

```
In [17]: # Display unique categories
sales["Category"].unique()
```

```
Out[17]: array(['Food', 'Beverages', 'Events', 'Liquor'], dtype=object)
```

Step 12 - Count Unique Categories

Count the number of unique categories in the dataset.

```
In [18]: # Count unique categories
sales["Category"].nunique()
```

```
Out[18]: 4
```

Step 13 - Count Unique Dates

Count the number of unique dates in the dataset.

```
In [19]: # Count unique dates
sales["Date"].nunique()
```

```
Out[19]: 13
```

Step 14 - Count Unique Sales Values

Count the number of unique sales values in the dataset.

```
In [20]: # Count unique sales values
sales["Sales"].nunique()
```

```
Out[20]: 52
```

Step 15 - Display Data Types

Display the data type of each column individually.

```
In [21]: # Display data types
sales.dtypes
```

```
Out[21]: Date          datetime64[ns]
Category          object
Sales             float64
dtype: object
```

Step 16 - Check for Negative Sales

Verify whether the dataset contains any negative sales values.

```
In [22]: # Check for negative sales values
(sales["Sales"] < 0).sum()
```

```
Out[22]: np.int64(0)
```

Step 17 - Check for Zero Sales

Verify whether the dataset contains sales values equal to zero.

```
In [23]: # Check for zero sales values
(sales["Sales"] == 0).sum()
```

```
Out[23]: np.int64(0)
```

Step 18 - Verify Date Range

Display the earliest and latest dates in the dataset.

```
In [24]: # Display minimum and maximum dates
sales["Date"].min(), sales["Date"].max()
```

```
Out[24]: (Timestamp('2025-03-01 00:00:00'), Timestamp('2026-03-01 00:00:00'))
```

Step 19 - Sort the Dataset by Date

Sort the dataset in chronological order.

```
In [25]: # Sort the dataset by date
sales = sales.sort_values(by="Date")
```

Step 20 - Reset the Index

Reset the DataFrame index after sorting.

```
In [26]: # Reset the DataFrame index
sales.reset_index(drop=True, inplace=True)
```

Step 21 - Preview the Clean Dataset

Display the first rows of the cleaned dataset.

```
In [27]: # Preview cleaned dataset
sales.head()
```

```
Out[27]:
```

	Date	Category	Sales
0	2025-03-01	Food	74471.064
1	2025-03-01	Beverages	12902.220
2	2025-03-01	Events	1957.200
3	2025-03-01	Liquor	14545.870
4	2025-04-01	Food	61905.340

Step 22 - Export the Clean Dataset

Export the cleaned dataset as a new Excel file for future analysis.

```
In [28]: # Export cleaned Sales dataset
sales.to_excel(
    "../data/cleaned/sales_cleaned.xlsx",
    index=False
)
```

Step 23 - Cleaning Summary

Display a summary of the data quality checks performed during the cleaning process.

```
In [29]: print("===== CLEANING SUMMARY =====")
```

```
print(f"Rows: {sales.shape[0]}")
print(f"Columns: {sales.shape[1]}")
print(f"Missing values: {sales.isnull().sum().sum()}")
print(f"Duplicate rows: {sales.duplicated().sum()}")
print(f"Negative sales: {(sales['Sales'] < 0).sum()}")
print(f"Zero sales: {(sales['Sales'] == 0).sum()}")
print(f>Date range: {sales['Date'].min().date()} to {sales['Date'].max().date()}")

print("Clean dataset exported successfully.")
print("=====")
```

===== CLEANING SUMMARY =====

Rows: 52

Columns: 3

Missing values: 0

Duplicate rows: 0

Negative sales: 0

Zero sales: 0

Date range: 2025-03-01 to 2026-03-01

Clean dataset exported successfully.

=====

Author

Guido Acosta

Data Analyst Portfolio Project

GitHub Portfolio – 2026